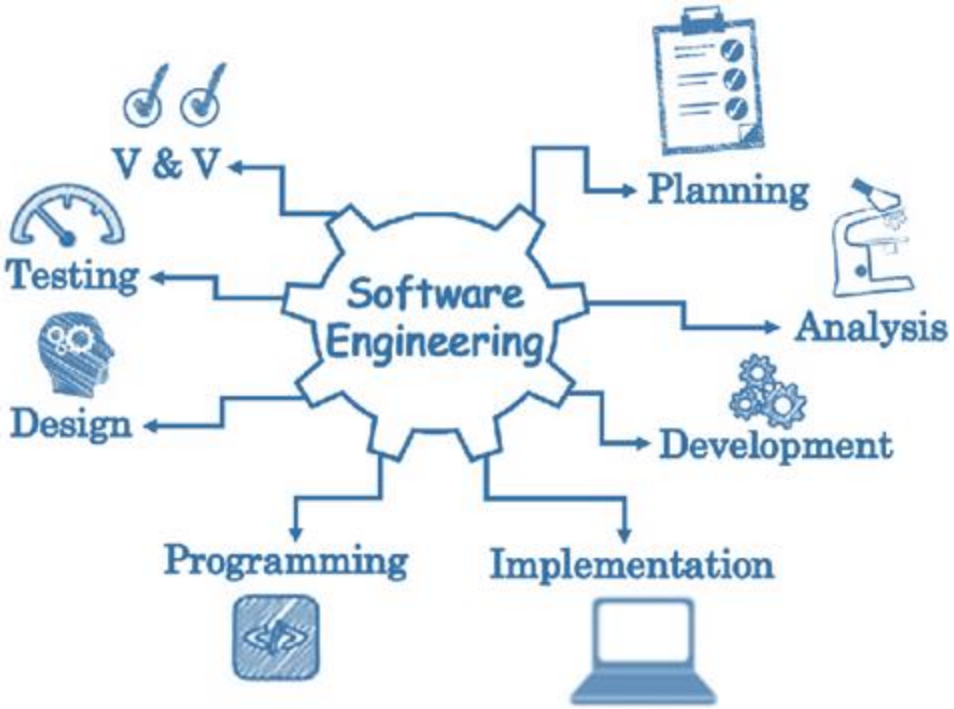
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Software Process Models and Project

4.1 OOAD Principles

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

4.2 Analysis Object Models and Dynamic Models

- Entity, Boundary, and Control Objects
- Generalization and Specialization

4.6 Case Study

- Applying OOAD and UML to a Real-World Application

4.3 UML Diagrams

- Use Case Diagrams
- Class Diagrams
- Sequence Diagrams
- Activity Diagrams
- State Machine Diagrams

4.4 Modeling Interactions and Object Lifecycles

- Mapping Use Cases to Objects
- Modeling Associations and Aggregations
- Modeling Inheritance Relationships

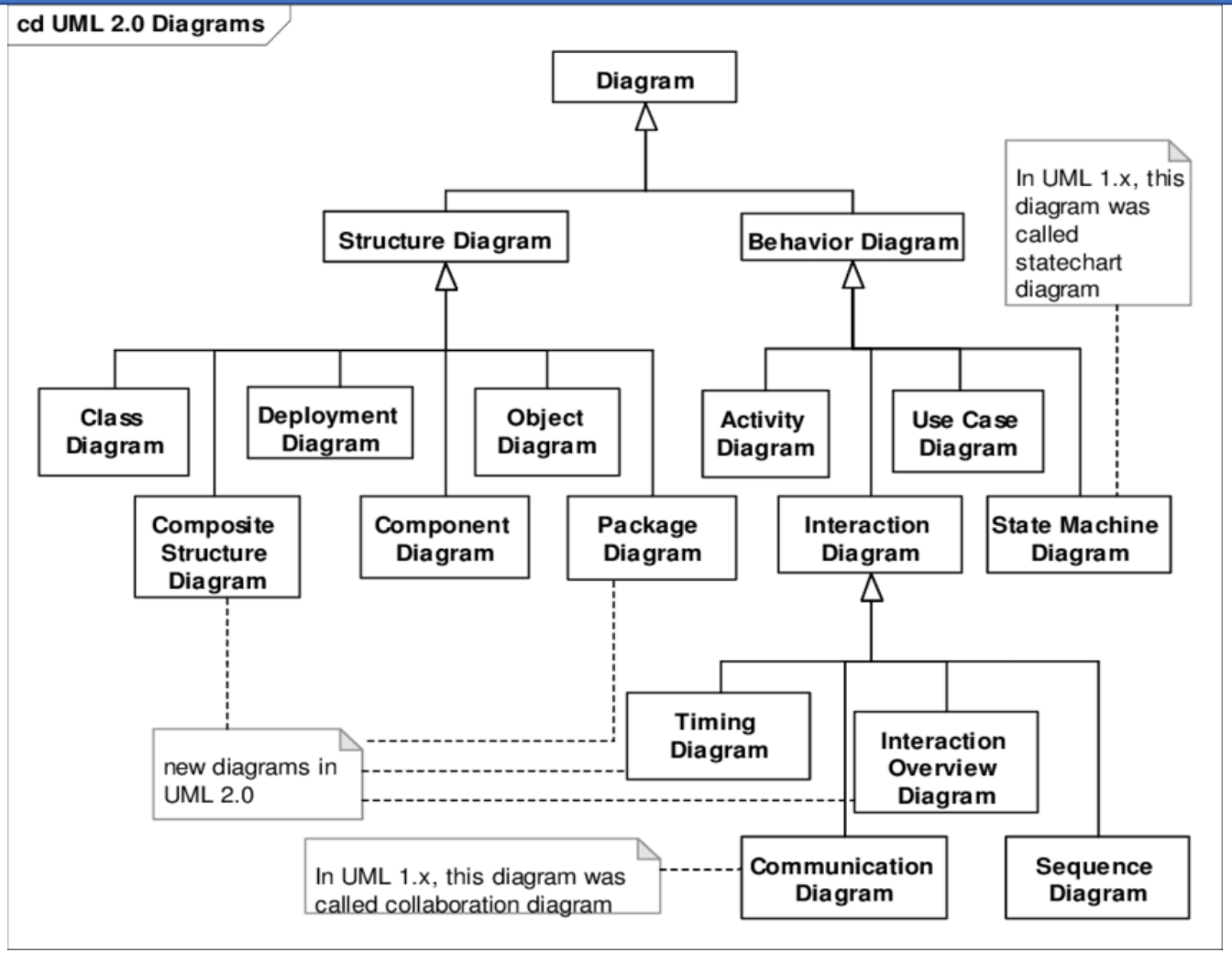
4.5 Reviewing Analysis and Design Models

- Iterative Refinement
- Client Sign-Off Procedures

UML Diagrams

The Unified Modeling Language (**UML**) is a language for specifying, constructing, visualizing, and documenting the artifacts of a software-intensive system.

Structure diagrams show the **static structure** of the system and its parts on different abstraction and implementation **levels** and how they are related to each other.
Behavior diagrams show the **dynamic behavior** of the objects in a system, which can be described as a series of changes to the system over **time**.



UML Diagrams

UML (Unified Modeling Language)

An emerging standard for modeling object-oriented software.

Resulted from the convergence of notations from three leading object-oriented methods:

OMT (James Rumbaugh)

OOSE (Ivar Jacobson)

Booch (Grady Booch)

Reference: "The Unified Modeling Language User Guide", Addison Wesley, 1999.

Supported by several CASE tools

Rational ROSE

TogetherJ

Rest of the Presentation slides are adopte from: Bernd Bruegge and Allen Dutoit.

UML Diagrams

UML Core Conventions

- Rectangles are classes or instances
- Ovals are functions or use cases
- Instances are denoted with an underlined names
  - `myWatch:SimpleWatch`
  - `Joe:Firefighter`
- Types are denoted with non underlined names
  - `SimpleWatch`
  - `Firefighter`
- Diagrams are graphs
  - Nodes are entities
  - Arcs are relationships between entities

UML Diagrams

Why model software?

- Software is getting increasingly more complex
  - Facebook uses 62 million lines of code. This is pretty huge.
  - The search engine giant Google uses around two billion code lines for all its services.
  - A single programmer cannot manage this amount of code in its entirety.
- Code is not easily understandable by developers who did not write it
- We need simpler representations for complex systems
  - Modeling is a mean for dealing with complexity

UML Diagrams

- Standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, business modeling and other non-software systems.
- The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.
- The UML is a very important part of developing object oriented software and the software development process.
- The UML uses mostly graphical notations to express the design of software projects.
- Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

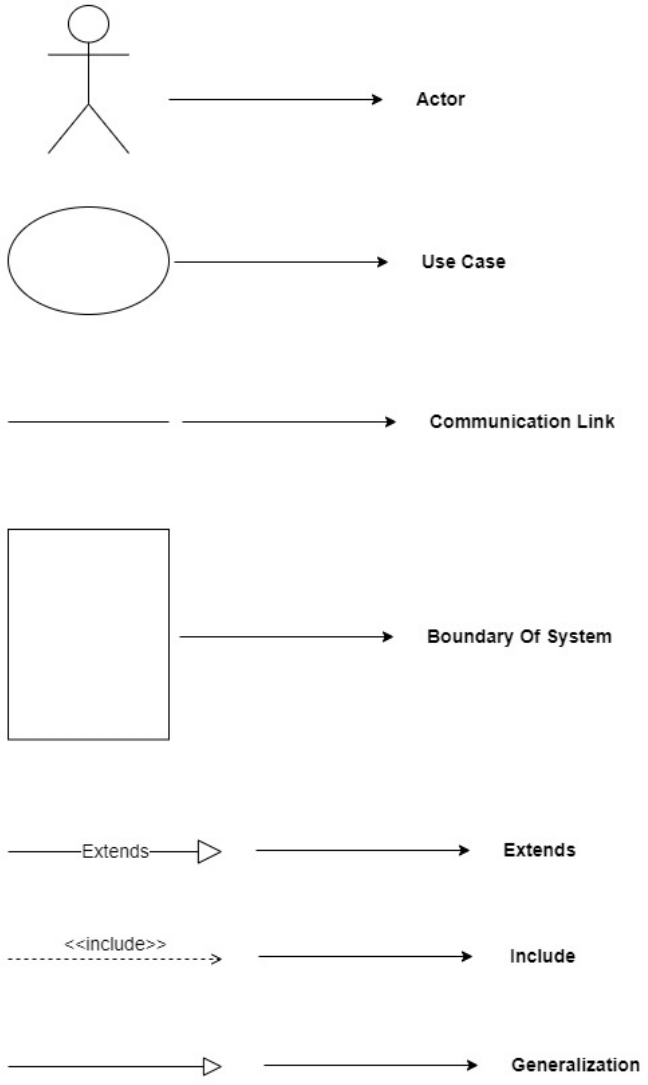
UML Diagrams

- Use case Diagrams
  - Describe the functional behavior of the system as seen by the user.
- Class diagrams
  - Describe the static structure of the system: Objects, Attributes, Associations
- Sequence diagrams
  - Describe the dynamic behavior between actors and the system and between objects of the system
- State Machine (State chart) diagrams
  - Describe the dynamic behavior of an individual object (essentially a finite state automaton)
- Activity Diagrams
  - Model the dynamic behavior of a system, in particular the workflow (essentially a flowchart)

UML Diagrams

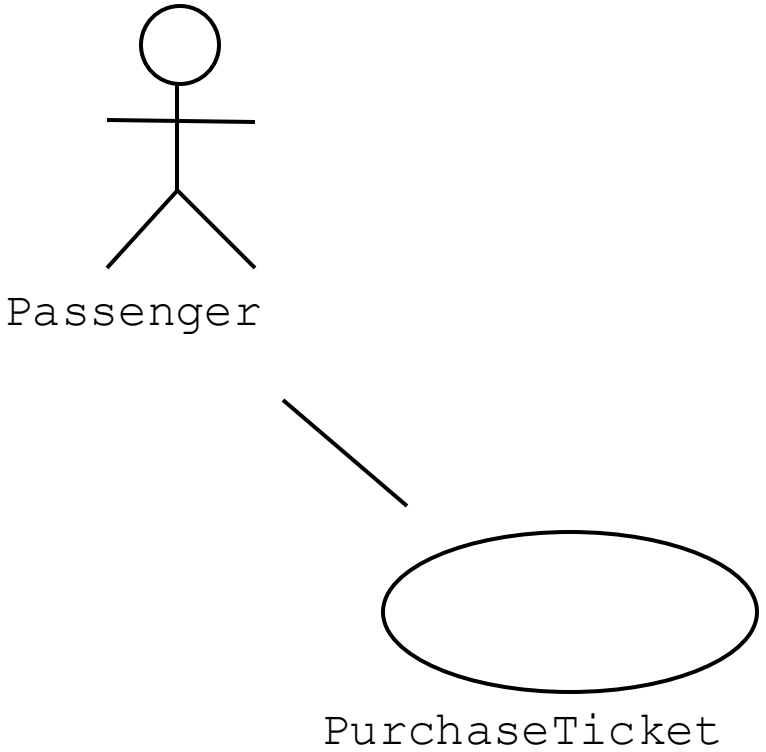
- **Use- Case Diagram**

- **When to Use?**
  - Use cases are used during requirements elicitation and analysis to represent the functionality of the system.
- **Focus?**
  - Use cases focus on the behavior of the system from an external point of view.
  - A use case describes a function provided by the system that yields a visible result for an actor.



UML Diagrams

- **Use- Case Diagram**



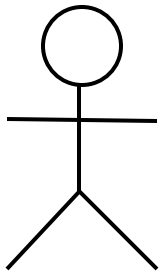
- Used during requirements elicitation to represent external behavior

- ***Actors*** represent roles, that is, a type of user of the system
- ***Use cases*** represent a sequence of interaction for a type of functionality
- The use case model is the set of all use cases. It is a complete description of the functionality of the system and its environment

UML Diagrams

- **Use- Case Diagram**

  Actors



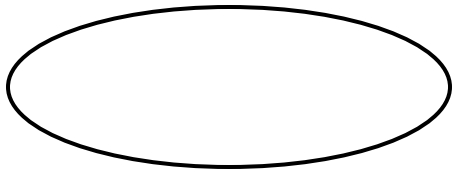
Passenger

- An actor models an external entity which communicates with the system:
  - User
  - External system
  - Physical environment
- An actor has a unique name and an optional description.
- Examples:
  - Passenger: A person in the train
  - GPS satellite: Provides the system with GPS coordinates

UML Diagrams

- **Use- Case Diagram**

  Use Case



PurchaseTicket

A use case represents a class of functionality provided by the system as an event flow.

A use case consists of:

- Unique name
- Participating actors
- Entry conditions
- Flow of events
- Exit conditions
- Special requirements

UML Diagrams

Use Case Diagram: Example

Name: `Purchase ticket`

Participating actor: `Passenger`

Entry condition:

- `Passenger` standing in front of ticket distributor.
- `Passenger` has sufficient money to purchase ticket.

Exit condition:

- `Passenger` has ticket.

Event flow:

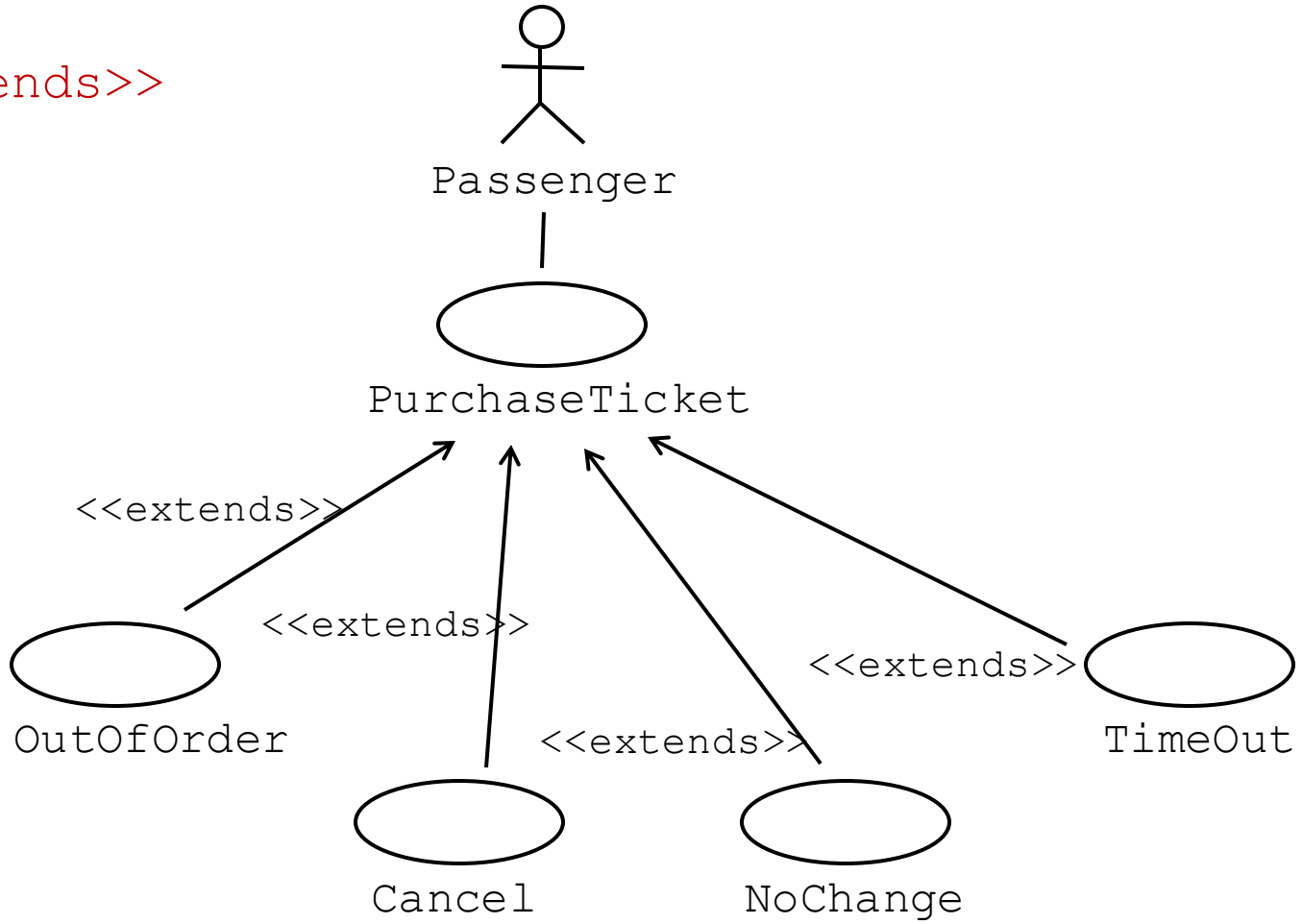
1. `Passenger` selects the number of zones to be traveled.
2. Distributor displays the amount due.
3. `Passenger` inserts money, of at least the amount due.
4. Distributor returns change.
5. Distributor issues ticket.

Anything missing?

Exceptional cases!

UML Diagrams

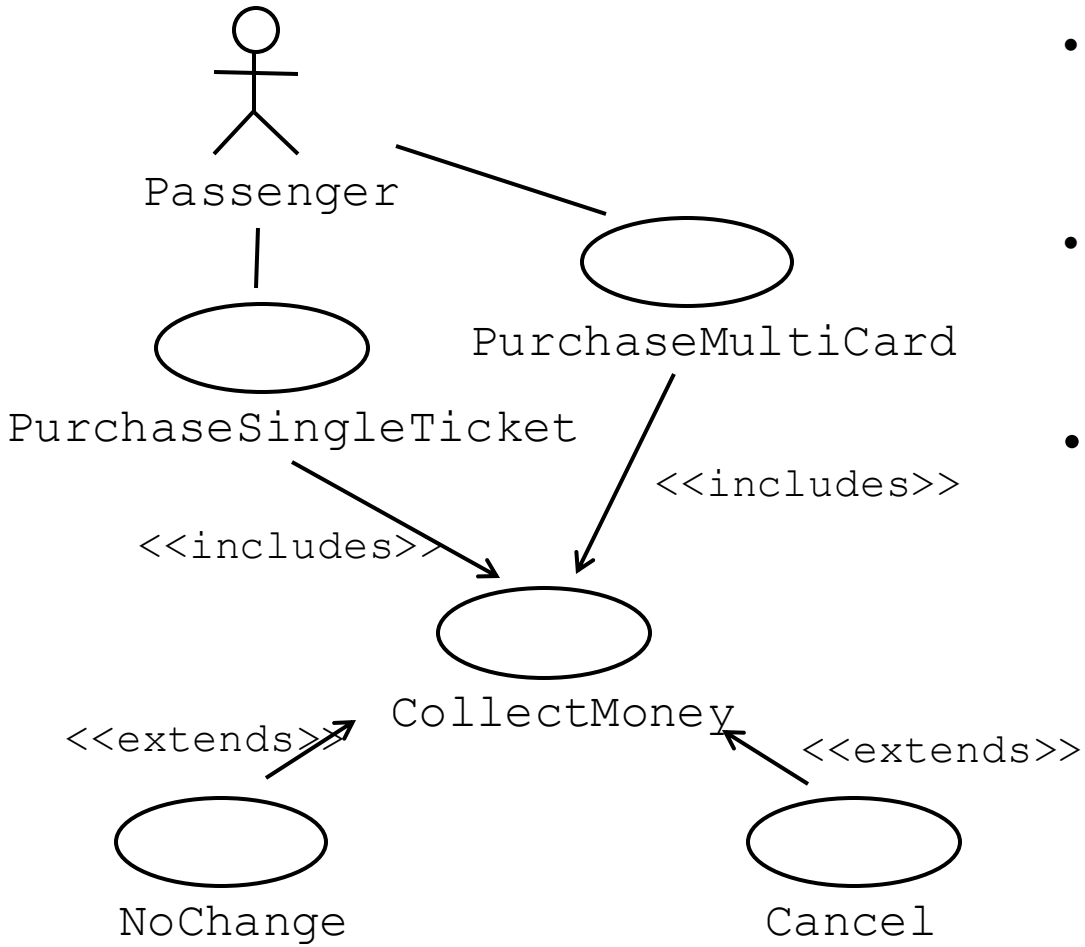
The <<extends>> Relationship



- <<extends>> relationships represent exceptional or seldom invoked cases.
- The exceptional event flows are factored out of the main event flow for clarity.
- Use cases representing exceptional flows can extend more than one use case.
- The direction of a <<extends>> relationship is to the extended use case

UML Diagrams

The <<includes>> Relationship



- <<includes>> relationship represents behavior that is factored out of the use case.
- <<includes>> behavior is factored out for reuse, not because it is an exception.
- The direction of a <<includes>> relationship is to the using use case (unlike <<extends>> relationships).

Class Diagram

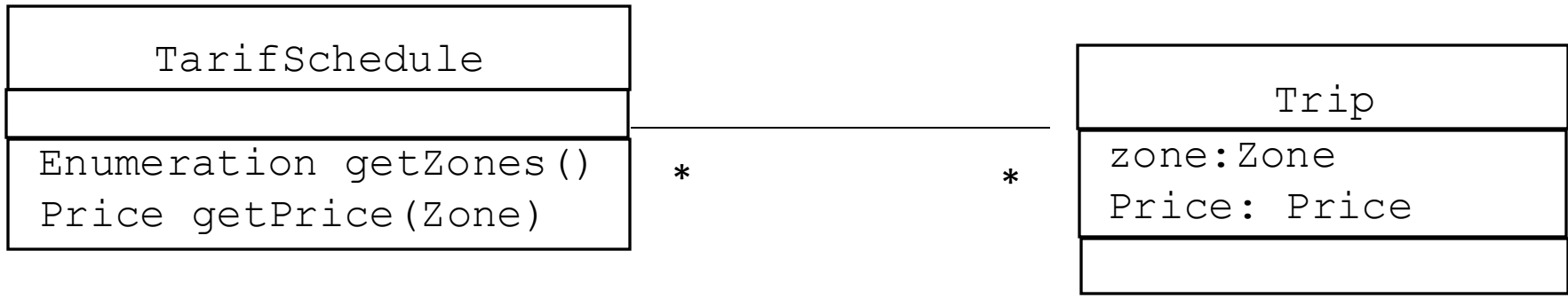
UML Diagrams

- **Class Diagram**

- A class diagram
  - expresses class definitions to be implemented
  - lists name, attributes, and methods for each class
  - shows relationships between classes
- UML allows different levels of detail on both the attributes and the methods of one class
  - could be just the the class name in a rectangle
  - or like the general form shown on the next slide

UML Diagrams

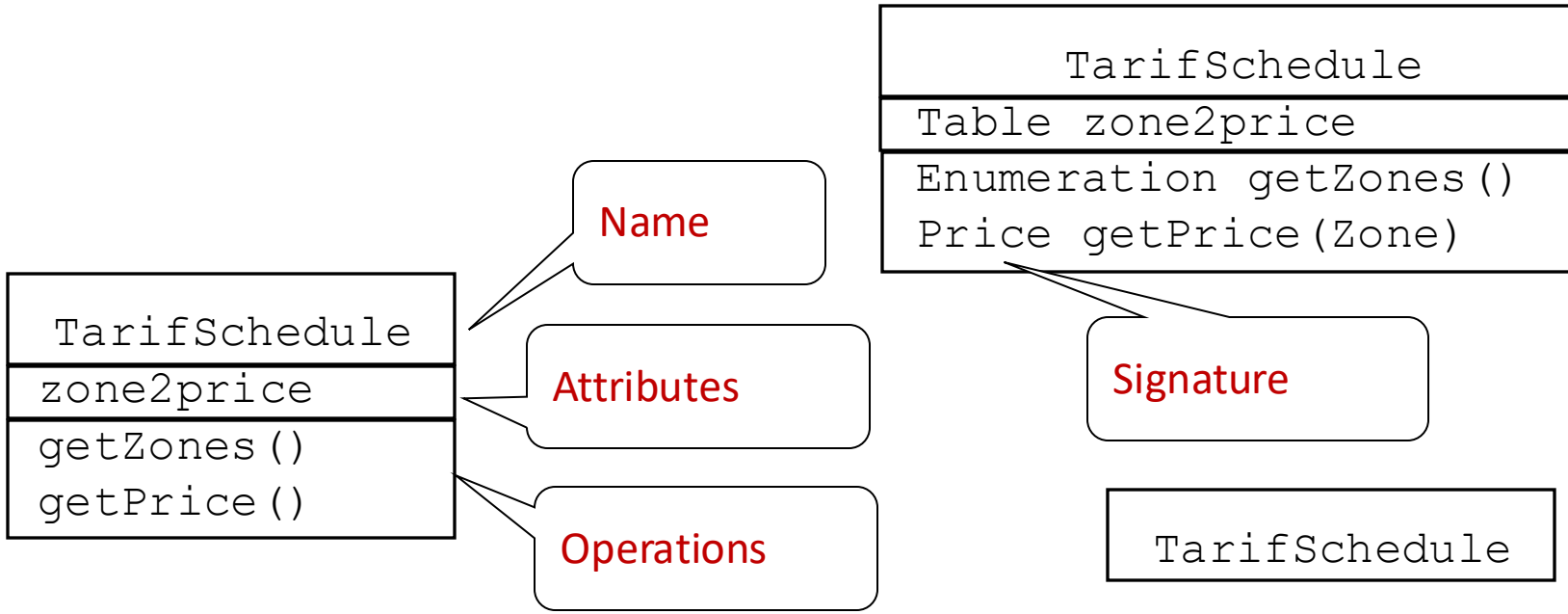
- **Class Diagram**



- Class diagrams represent the structure of the system.
- Used
  - during requirements analysis to model problem domain concepts
  - during system design to model subsystems and interfaces
  - during object design to model classes.

UML Diagrams

- **Class Diagram**

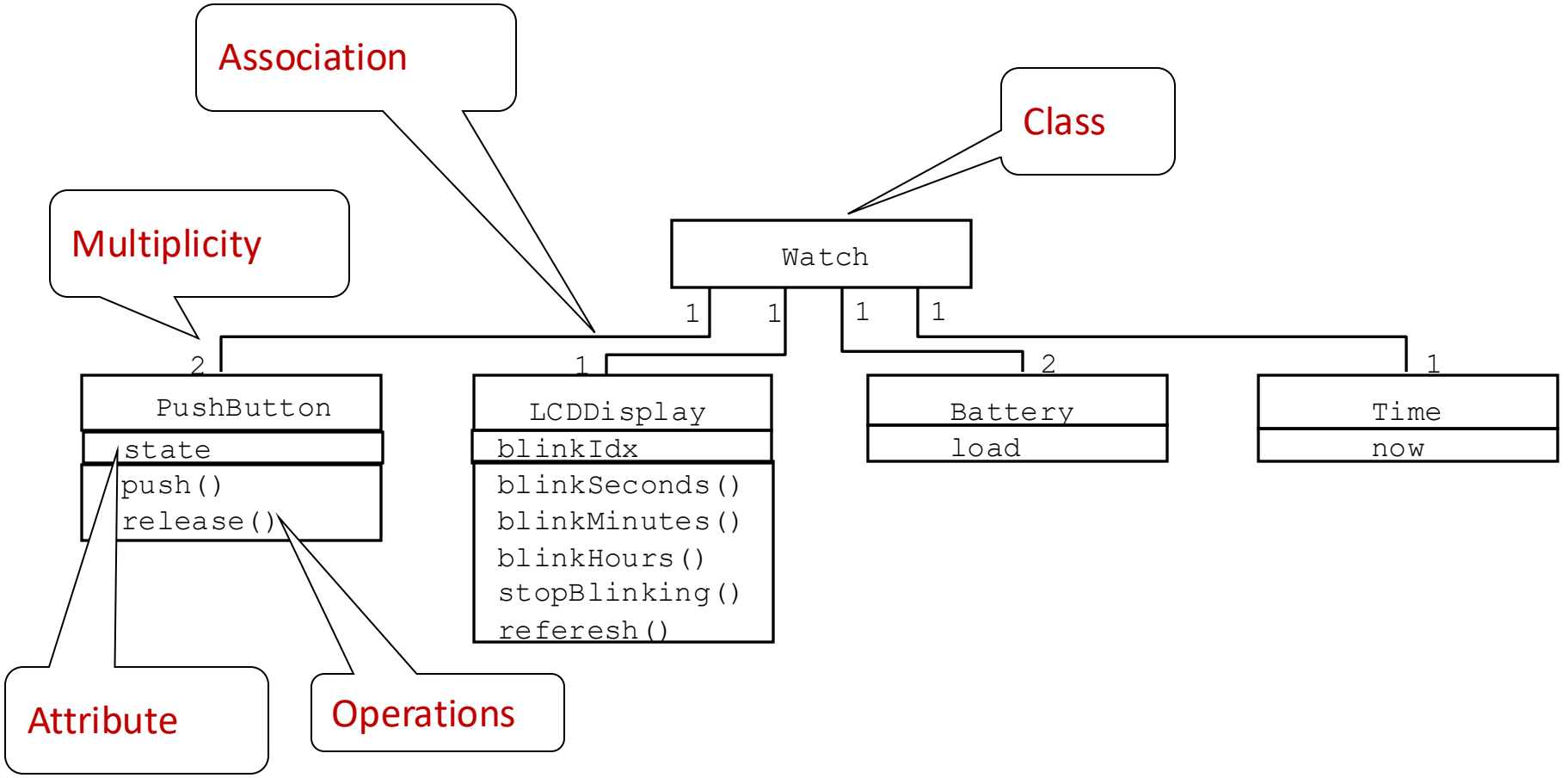


- A ***class*** represent a concept
- A class encapsulates state ***(attributes)*** and behavior ***(operations).***
- Each attribute has a ***type***.
- Each operation has a ***signature***.
- The class name is the only mandatory information.

UML Diagrams

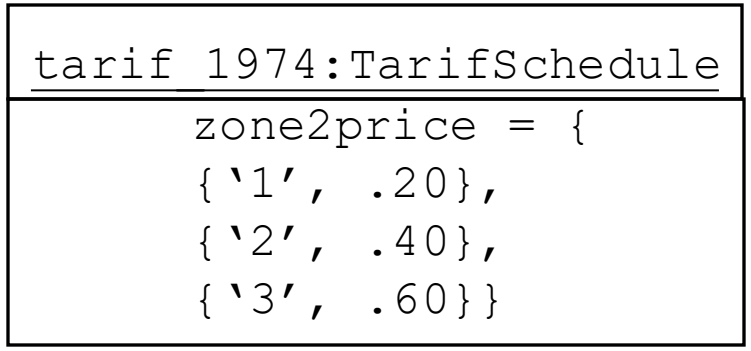
- **Class Diagram**

Class diagrams represent the structure of the system



UML Diagrams

- **Class Diagram**
- *Instance*



- An ***instance*** represents a phenomenon.
- The name of an instance is underlined and can contain the class of the instance.
- The attributes are represented with their ***values***.

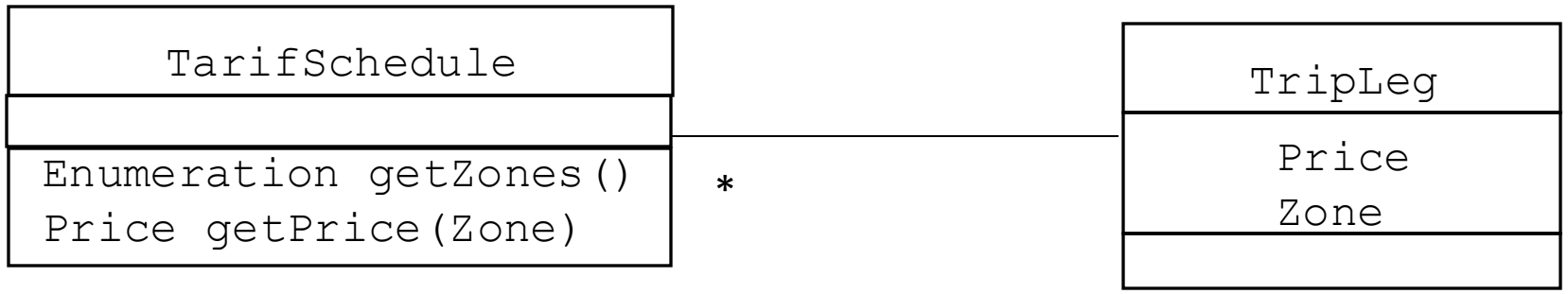
UML Diagrams

- **Class Diagram**
- *Actor Vs Instance*

- What is the difference between an *actor* , a *class* and an *instance*?
- Actor:
  - An entity outside the system to be modeled, interacting with the system (“Passenger”)
- Class:
  - An abstraction modeling an entity in the problem domain, must be modeled inside the system (“User”)
- Object:
  - A specific instance of a class (“Joe, the passenger who is purchasing a ticket from the ticket distributor”).

UML Diagrams

- **Class Diagram**
- *Association*

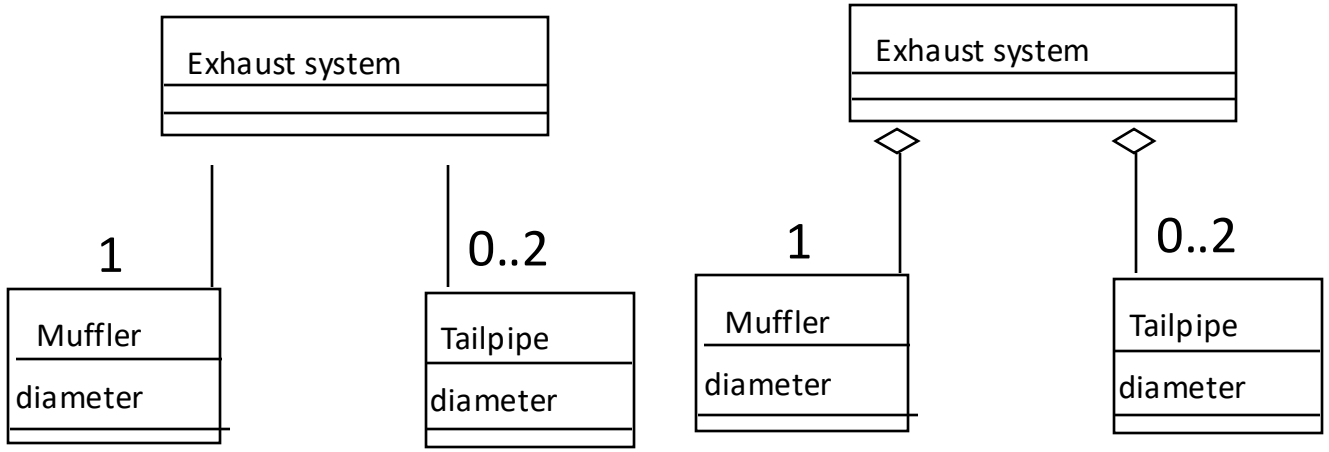


- Associations denote relationships between classes.
- The multiplicity of an association end denotes how many objects the source object can legitimately reference.

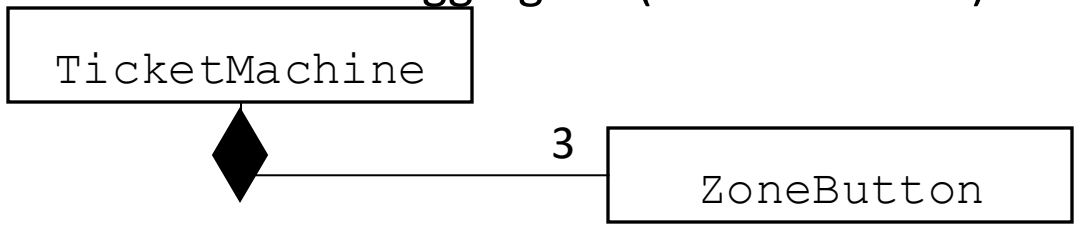
UML Diagrams

- **Class Diagram**
- *Aggregation*

- An ***aggregation*** is a special case of association denoting a “consists of” hierarchy.
- The ***aggregate*** is the parent class, the ***components*** are the children class.

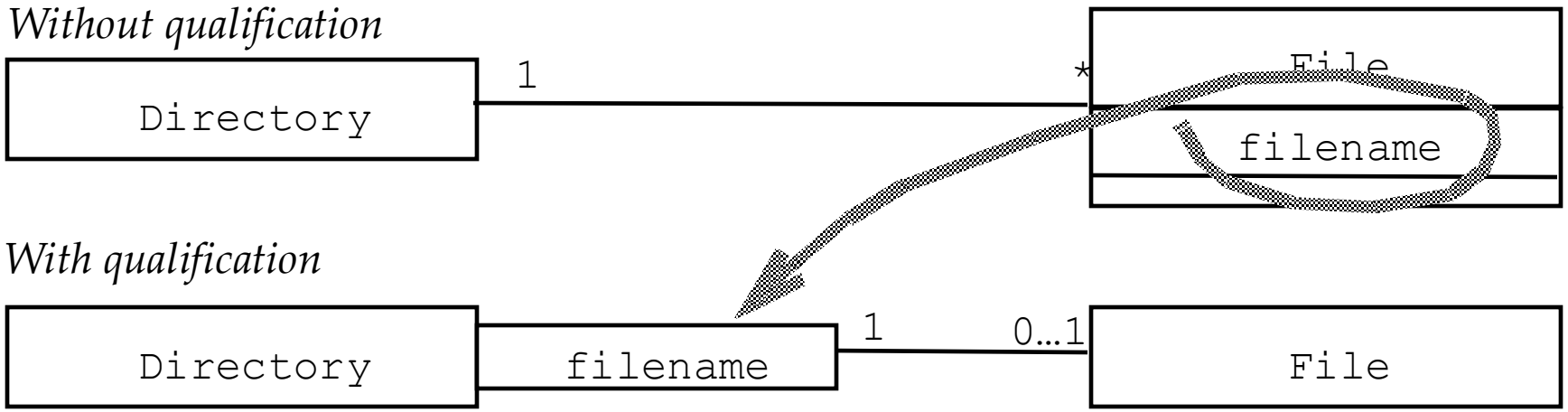


- A solid diamond denotes ***composition***, a strong form of aggregation where components cannot exist without the aggregate. (Bill of Material)



UML Diagrams

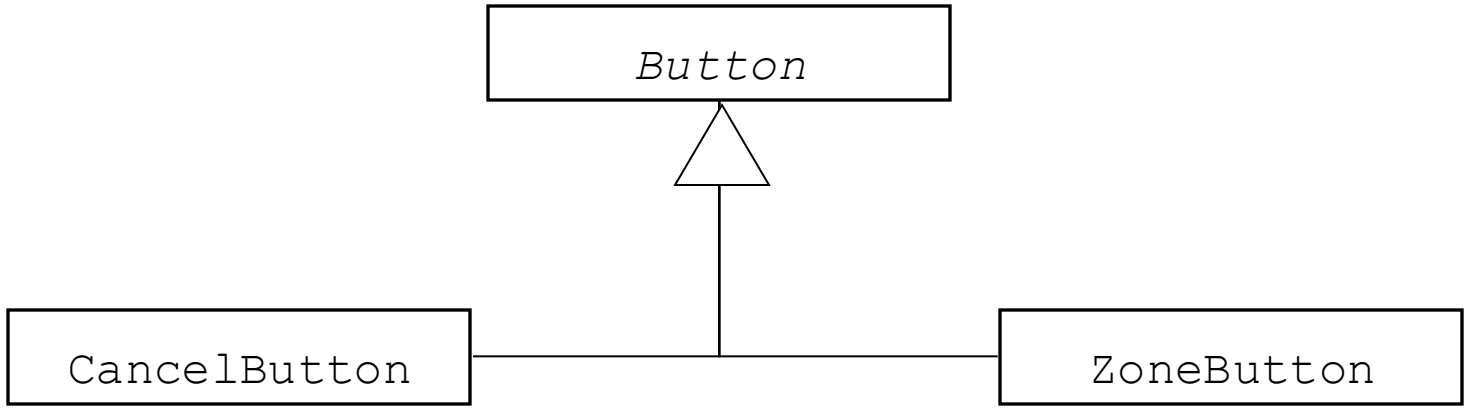
- **Class Diagram**
- *Qualifier*



- Qualifiers can be used to reduce the multiplicity of an association.

UML Diagrams

- **Class Diagram**
- *Inheritance*

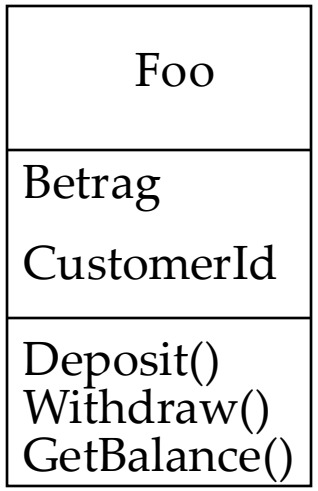


- The **children classes** inherit the attributes and operations of the **parent class**.
- Inheritance simplifies the model by eliminating redundancy.

UML Diagrams

Object Modeling in Practice: Class Identification

- **Class Diagram**
- *Object Modeling*

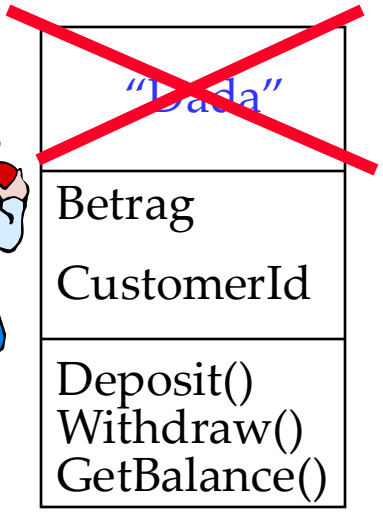


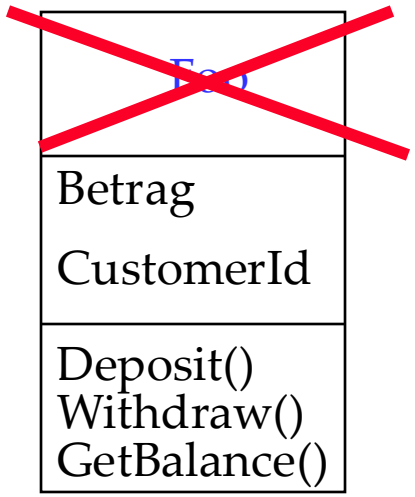
Class Identification: Name of Class, Attributes and Methods

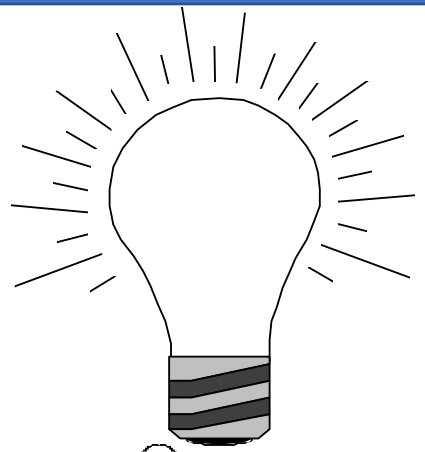
UML Diagrams

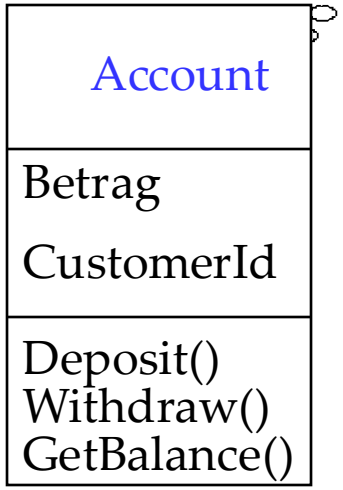
- **Class Diagram**
- *Object Modeling*







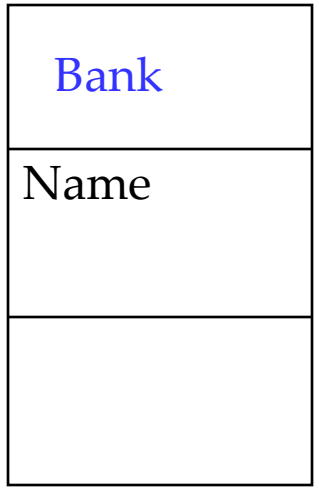


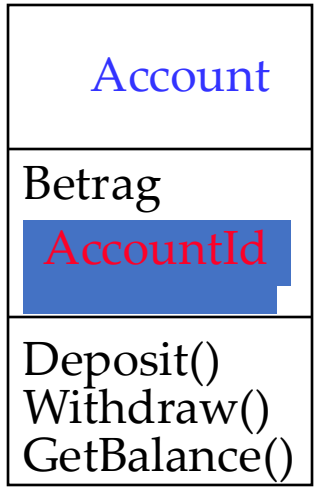


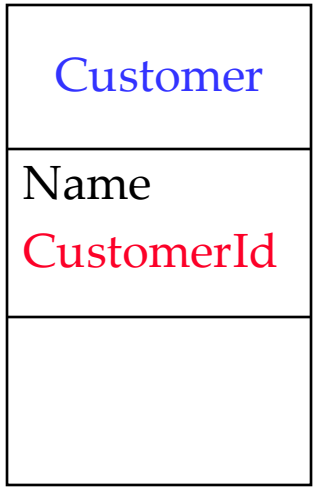
Naming is important!
Is Foo the right name?

UML Diagrams

- **Class Diagram**
- *Object Modeling*







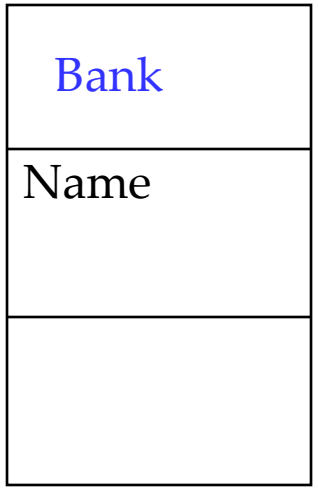
1) Find New Objects

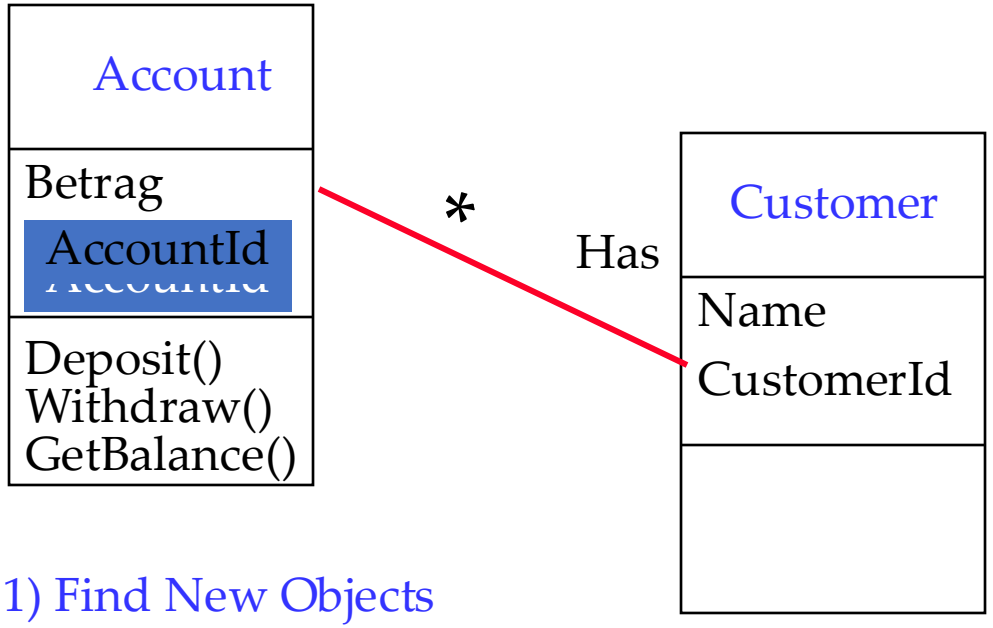
2) Iterate on Names, Attributes and Methods

UML Diagrams

- **Class Diagram**
- *Object Modeling*

Object Modeling in Practice: A Banking System





1) Find New Objects

2) Iterate on Names, Attributes and Methods

3) Find Associations between Objects

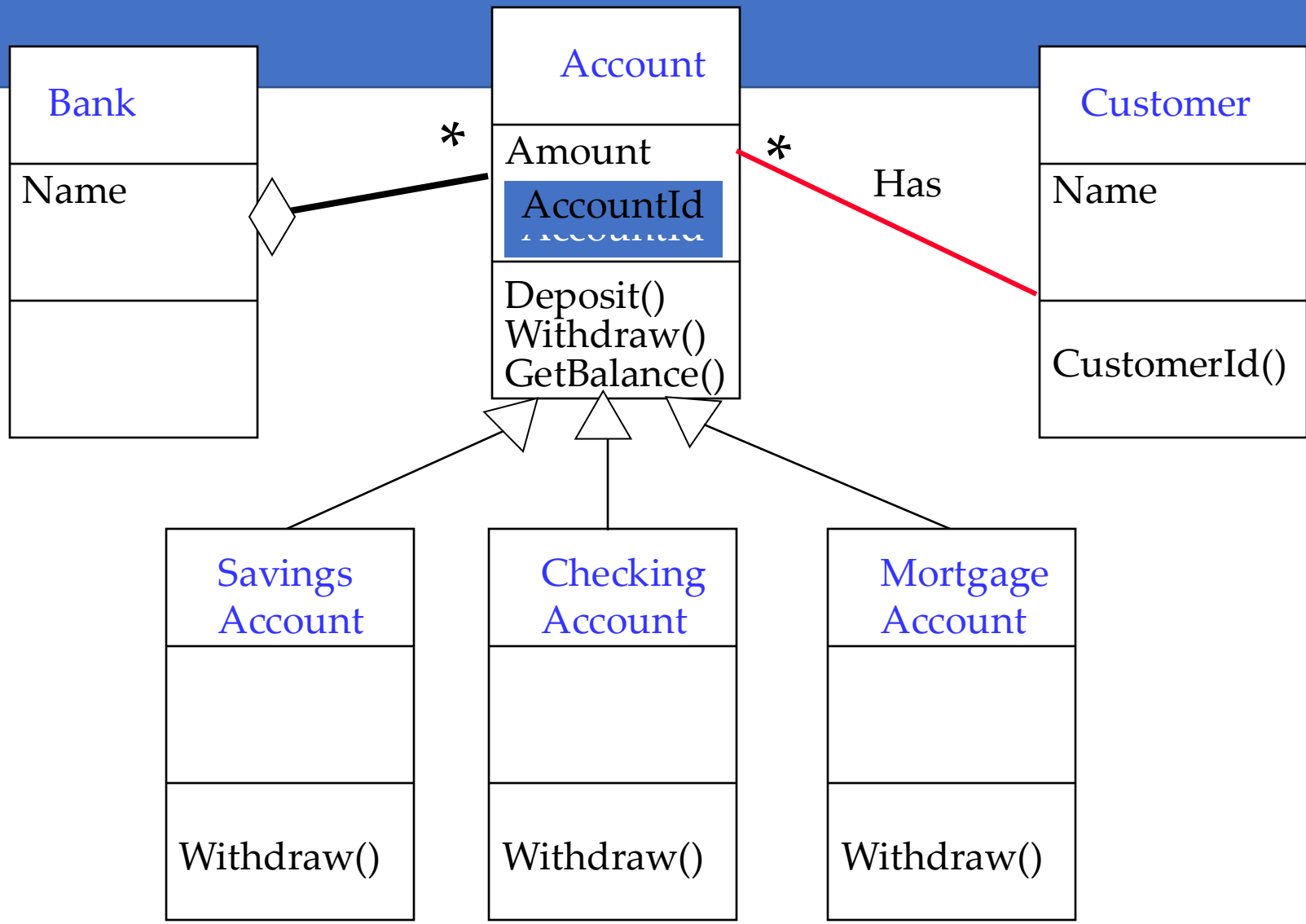
4) Label the assocations

5) Determine the multiplicity of the assocations

UML Diagrams

- **Class Diagram**
- *Object Modeling*

Practice Object Modeling: Iterate, Categorize!



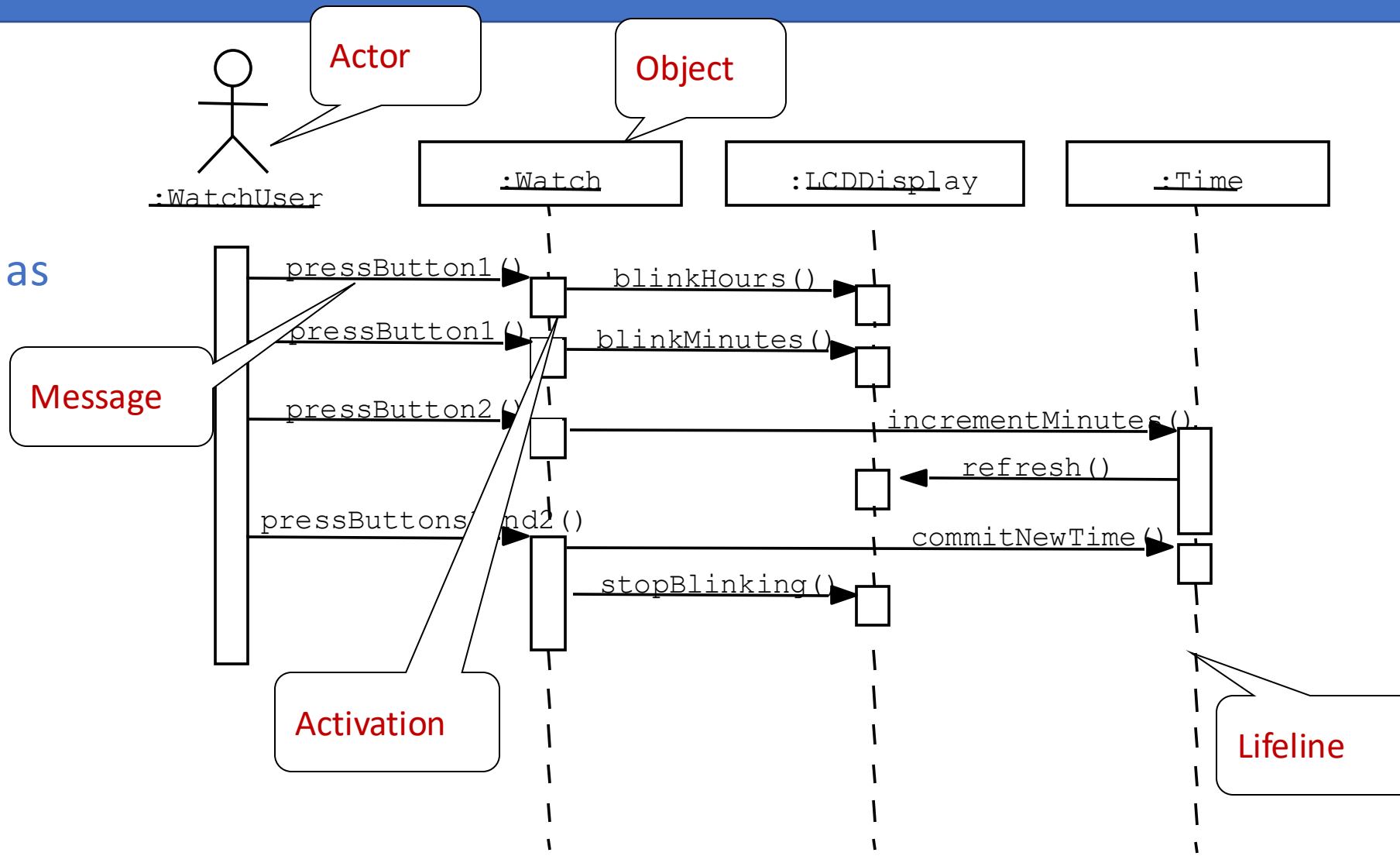
Sequence Diagram

UML Diagrams

- **Sequence Diagram**

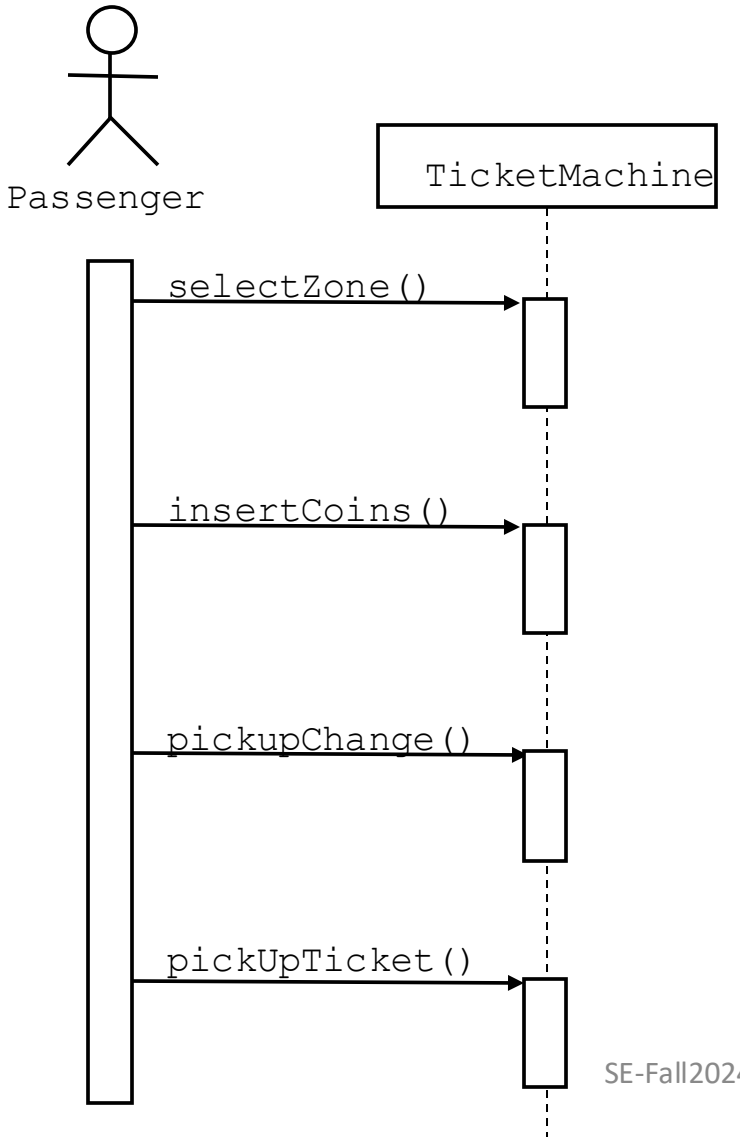
Sequence diagrams represent the behavior as interactions

Sequence diagrams represent the objects participating in the interaction horizontally and time vertically.



UML Diagrams

- **Sequence Diagram**

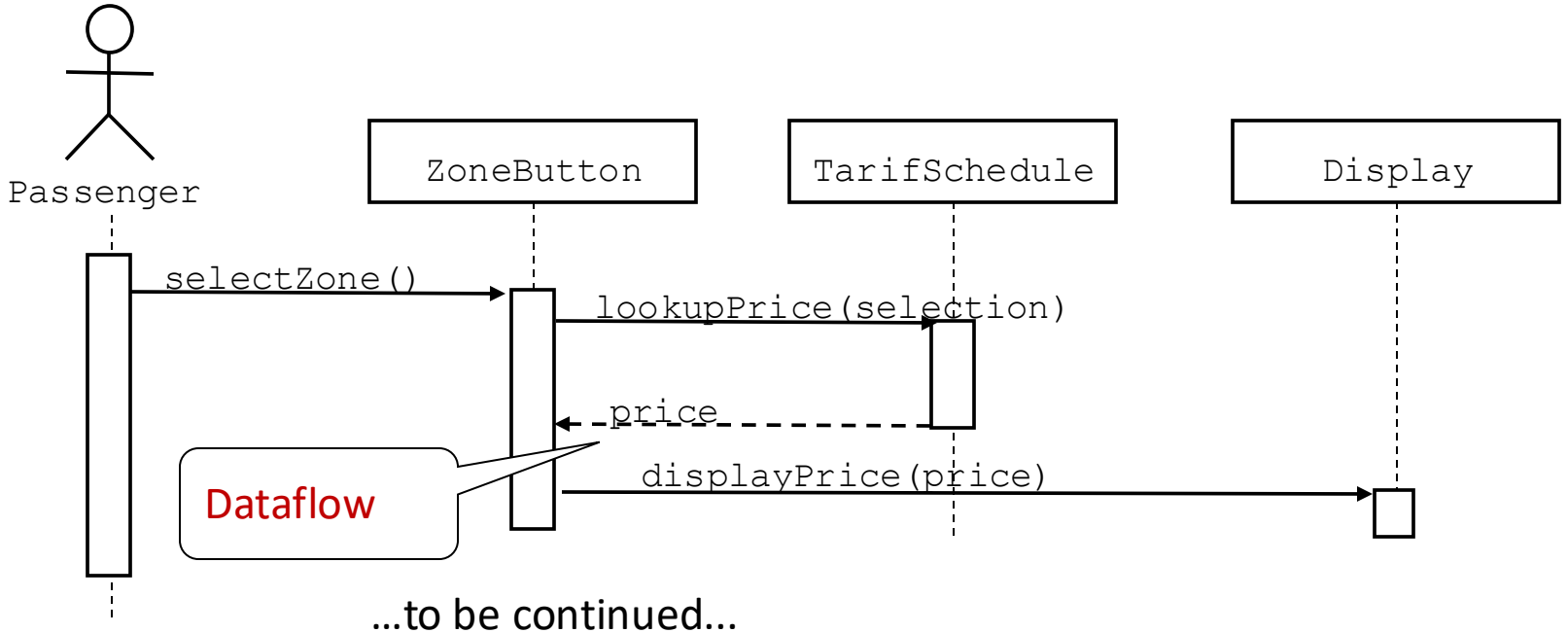


- Used during requirements analysis
  - To refine use case descriptions
  - to find additional objects (“participating objects”)
- Used during system design
  - to refine subsystem interfaces
- ***Classes*** are represented by columns
- ***Messages*** are represented by arrows
- ***Activations*** are represented by narrow rectangles
- ***Lifelines*** are represented by dashed lines

UML Diagrams

- **Sequence Diagram**

  Nested messages



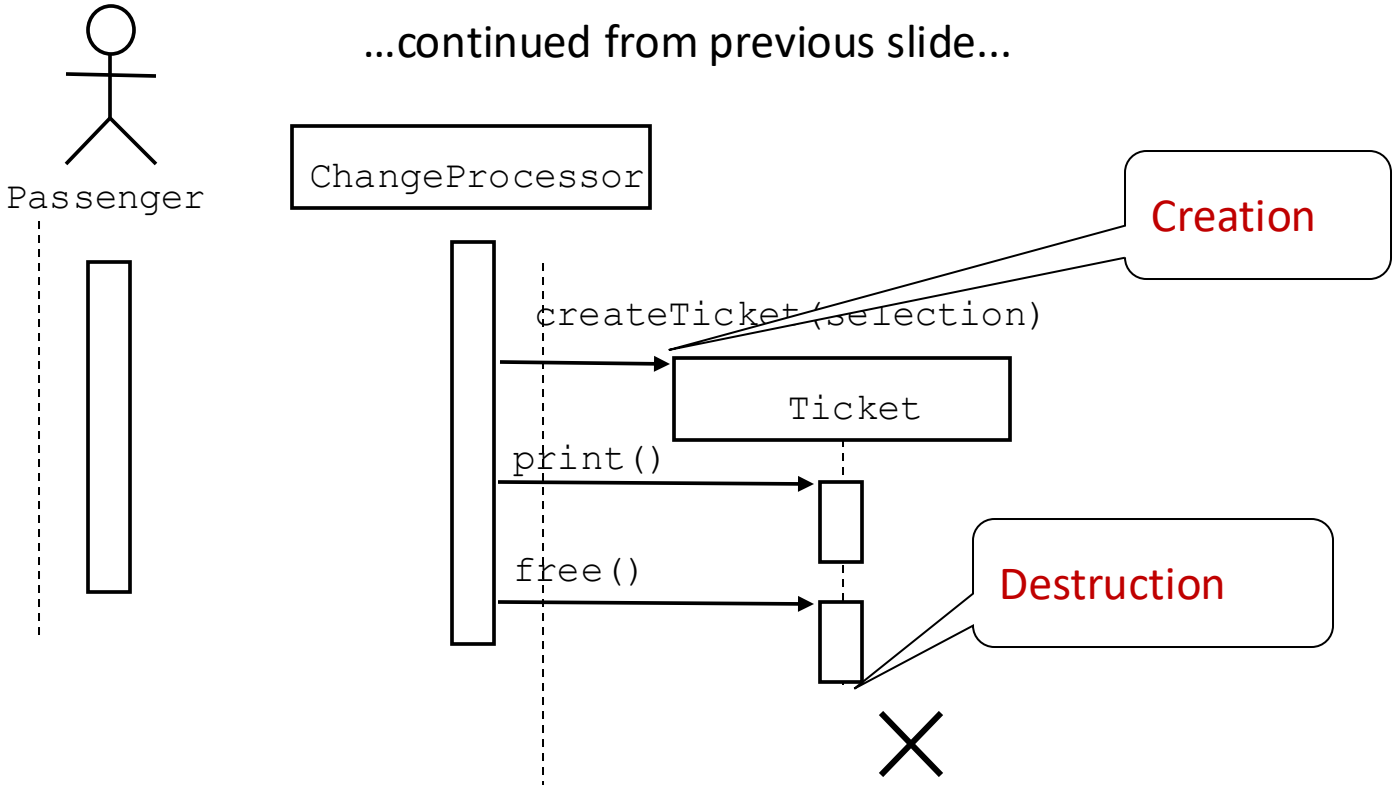
- The source of an arrow indicates the activation which sent the message
- An activation is as long as all nested activations
- Horizontal dashed arrows indicate data flow
- Vertical dashed lines indicate lifelines

UML Diagrams

- **Sequence Diagram**

  Iteration & condition

- Creation is denoted by a message arrow pointing to the object.
- Destruction is denoted by an X mark at the end of the destruction activation.
- In garbage collection environments, destruction can be used to denote the end of the useful life of an object.



UML Diagrams

- **Sequence Diagram**

Summary

- UML sequence diagram represent behavior in terms of interactions.
- Useful to find missing objects.
- Time consuming to build but worth the investment.
- Complement the class diagrams (which represent structure).

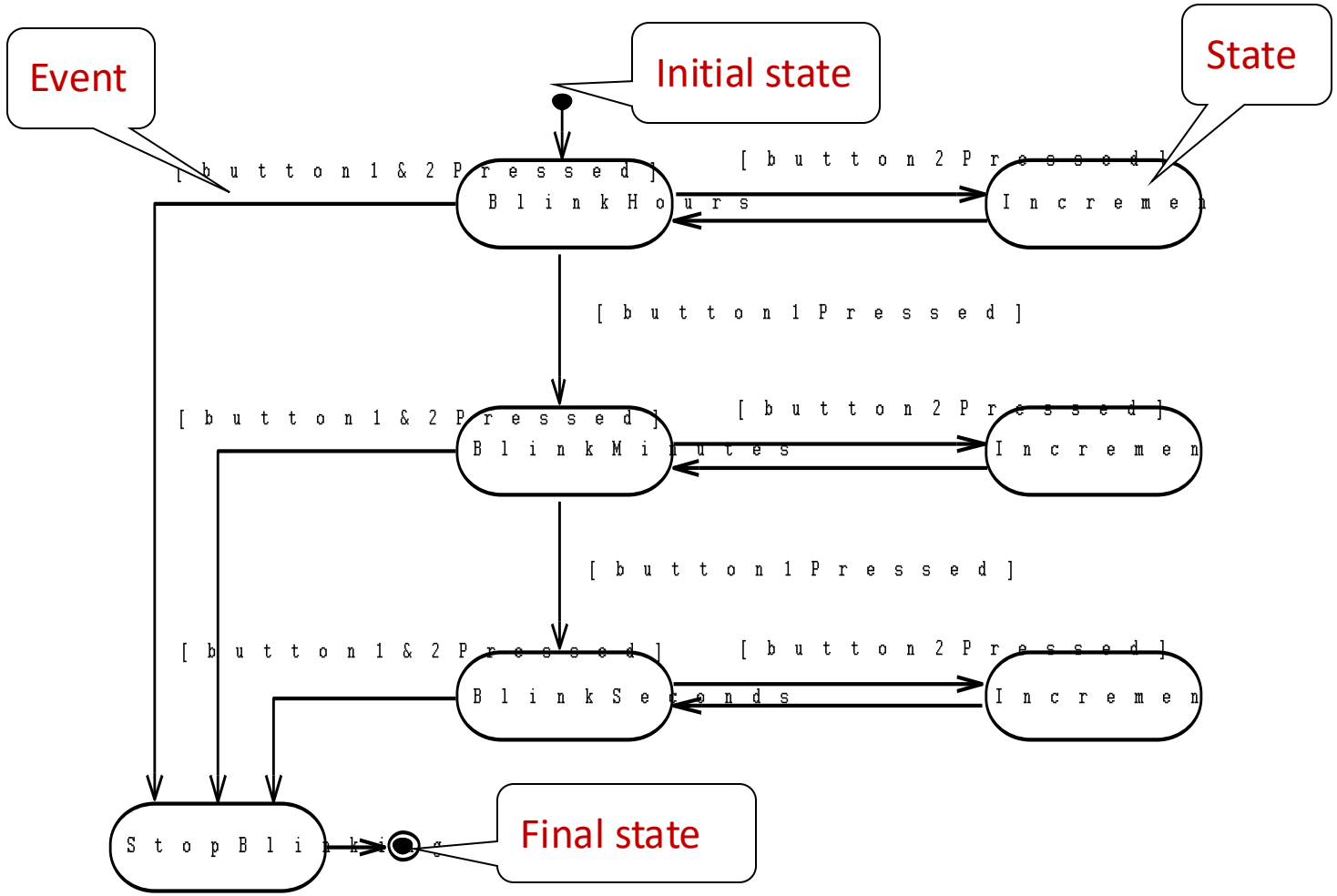
State Machine Diagram
Activity Diagram

UML Diagrams

- **State Machine Diagram**

State machine diagrams for objects with interesting dynamic behavior.

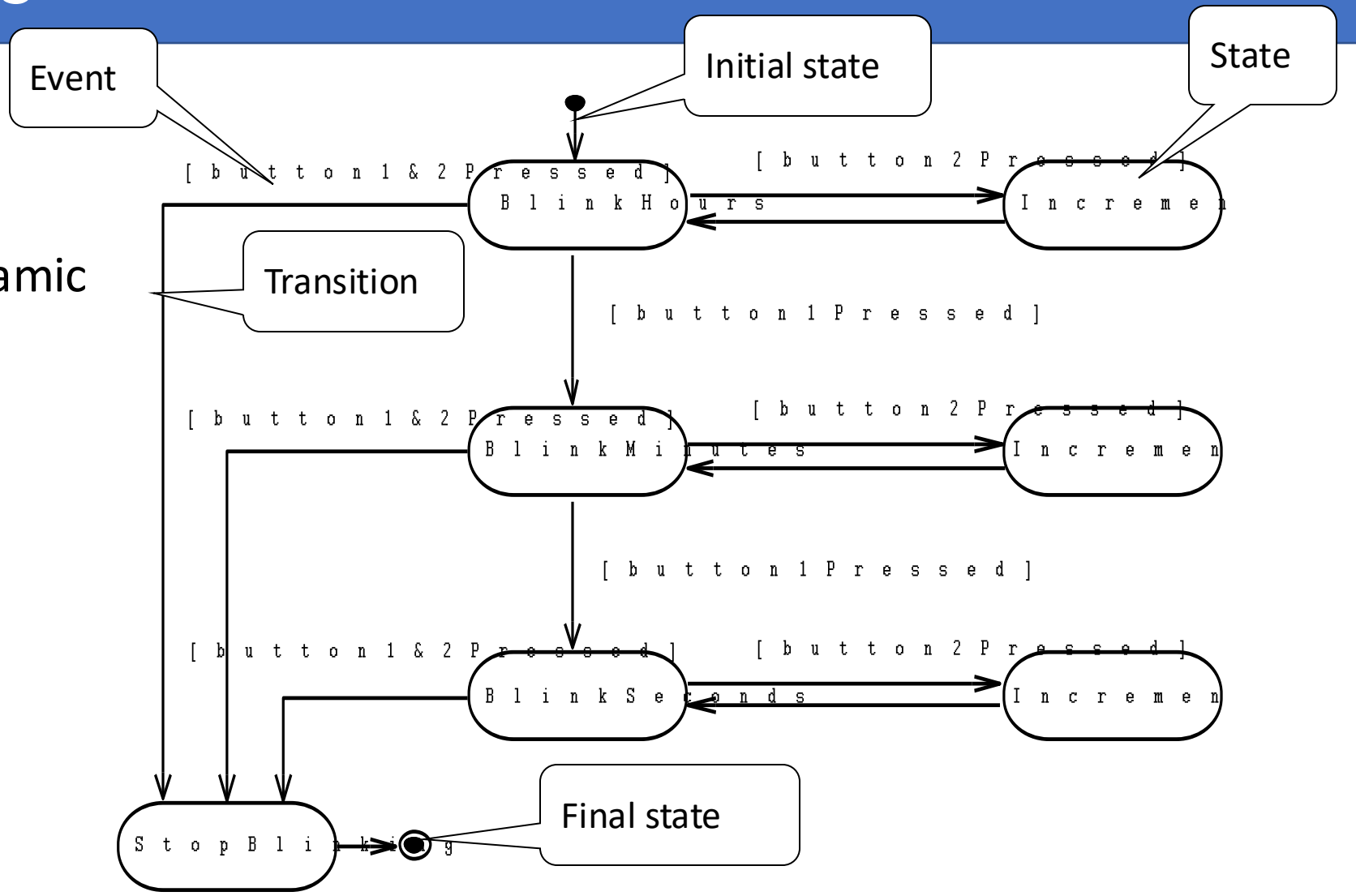
Represent behavior as states and transitions



UML Diagrams

- **State Machine Diagram**

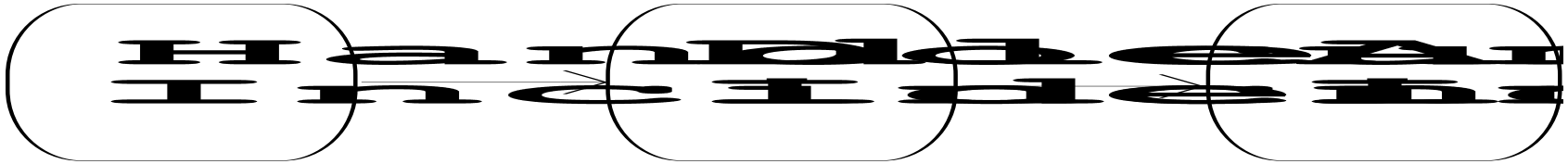
State machine diagrams for objects with interesting dynamic behavior.



UML Diagrams

- **Activity Diagram**

- An activity diagram shows flow control within a system

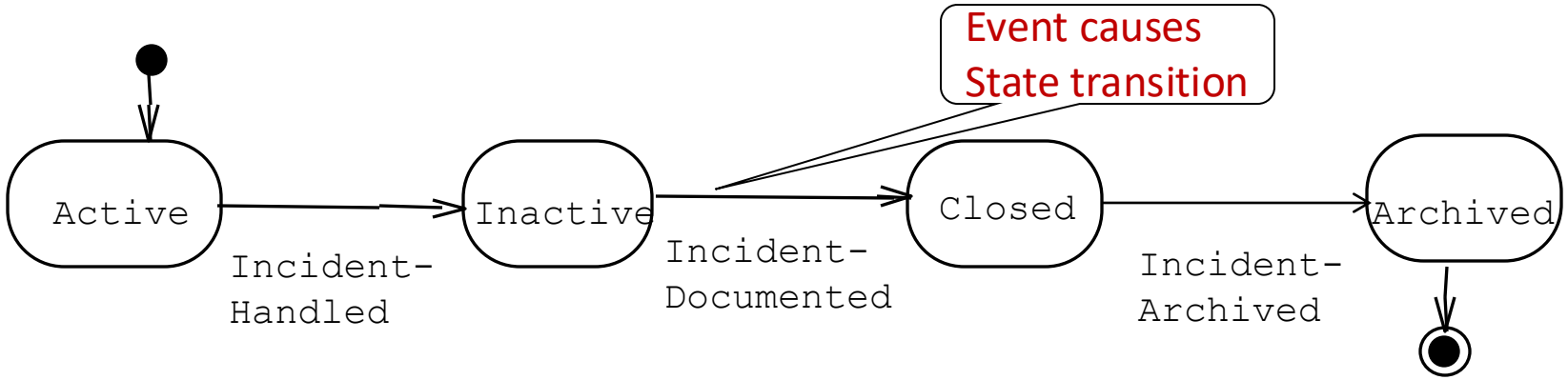


- An activity diagram is a special case of a state chart diagram in which states are activities (“functions”)
- Two types of states:
  - *Action state:*
    - Cannot be decomposed any further
    - Happens “instantaneously” with respect to the level of abstraction used in the model
  - *Activity state:*
    - Can be decomposed further
    - The activity is modeled by another activity diagram

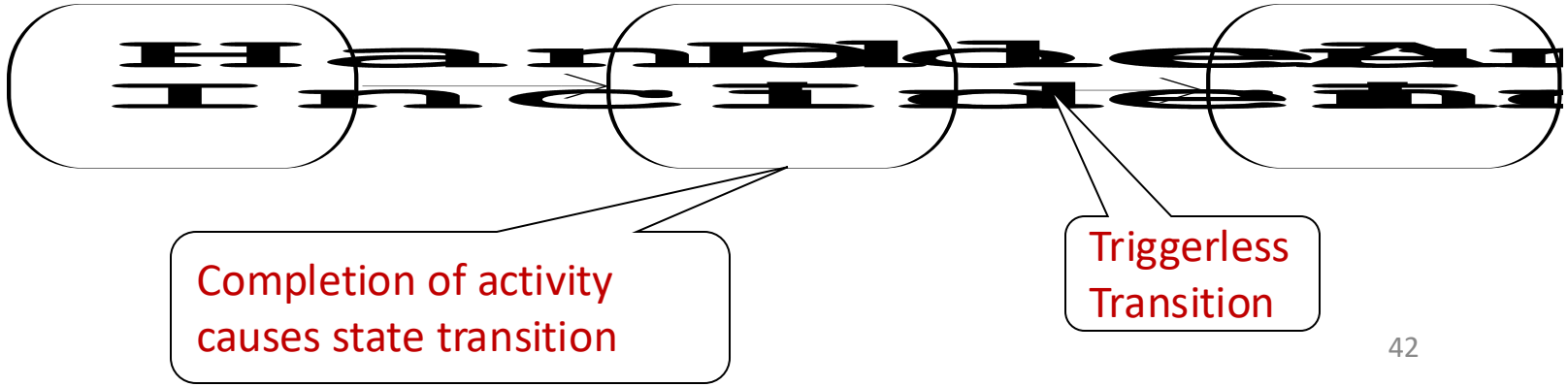
UML Diagrams

Statechart Diagram vs. Activity Diagram

Statechart Diagram for Incident (similar to Mealy Automaton)
(State: Attribute or Collection of Attributes of object of type Incident)

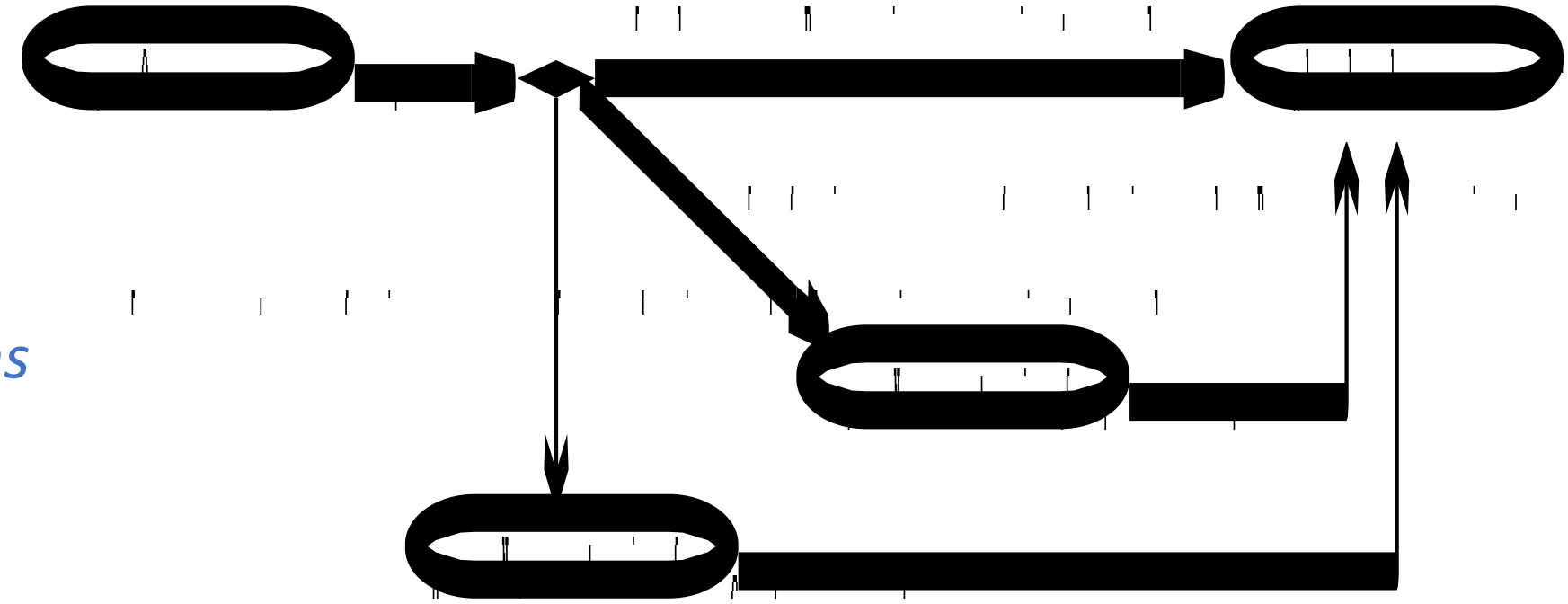


Activity Diagram for Incident (similar to Moore
(State: Operation or Collection of Operations)



UML Diagrams

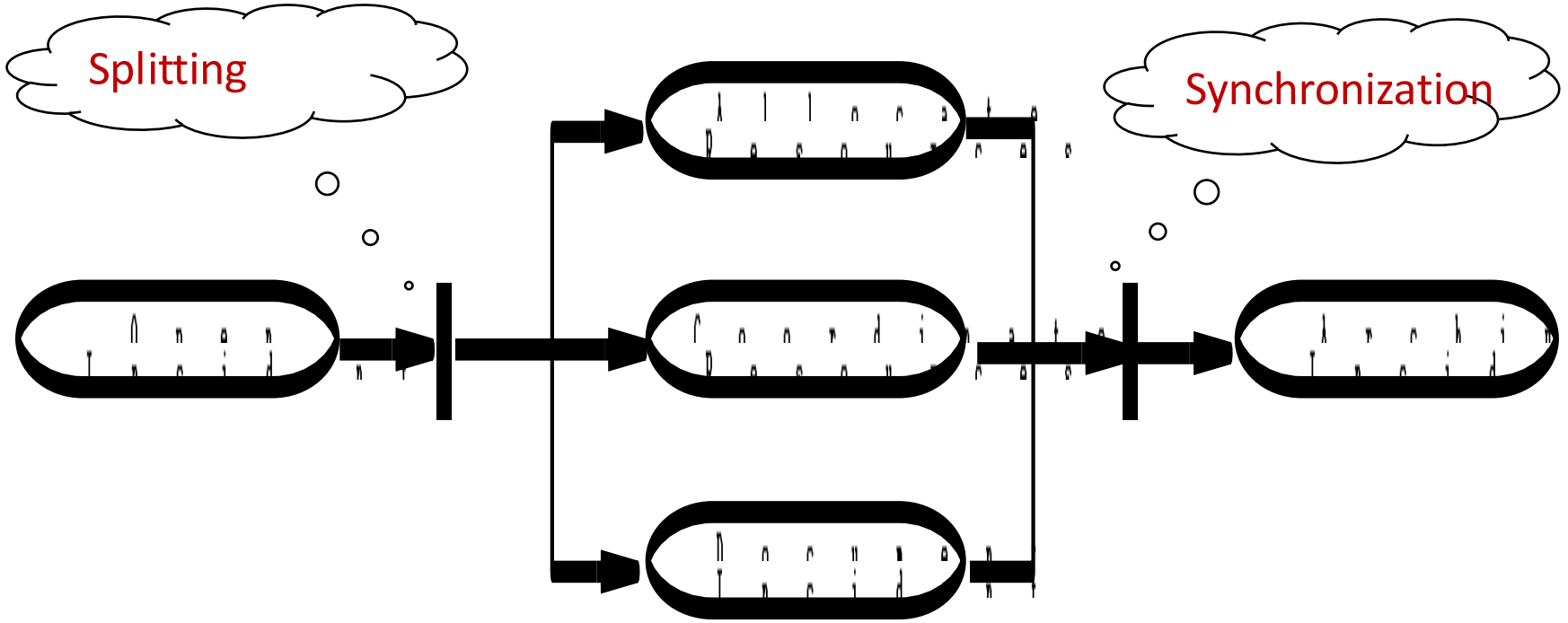
Activity Diagram: Modeling Decisions



UML Diagrams

Activity Diagrams:
Modeling Concurrency

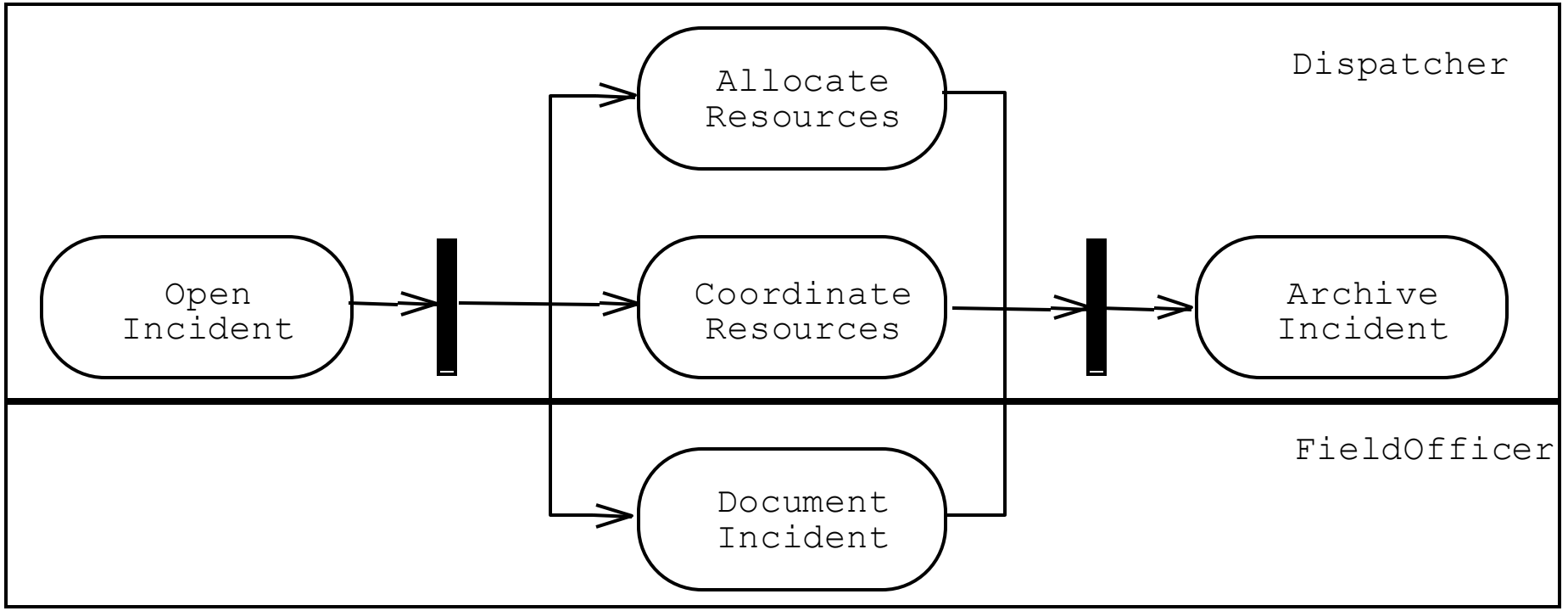
- Synchronization of multiple activities
- Splitting the flow of control into multiple threads



UML Diagrams

Activity Diagrams: Swimlanes

- Actions may be grouped into swimlanes to denote the object or subsystem that implements the actions.



UML Diagrams

What should be done first? Coding or Modeling?

- It all depends….
- Forward Engineering:
  - Creation of code from a model
  - Greenfield projects
- Reverse Engineering:
  - Creation of a model from code
  - Interface or reengineering projects
- Roundtrip Engineering:
  - Move constantly between forward and reverse engineering
  - Useful when requirements, technology and schedule are changing frequently

UML Diagrams

UML Summary

- UML provides a wide variety of notations for representing many aspects of software development
  - Powerful, but complex language
  - Can be misused to generate unreadable models
  - Can be misunderstood when using too many exotic features

- For now we concentrate on a few notations:
  - Functional model: Use case diagram
  - Object model: class diagram
  - Dynamic model: sequence diagrams, statechart and activity diagrams

UML Diagrams

Class Work

- Draw a use case diagram for a ticket distributor for a train system. The system includes two actors: a traveler who purchases different types of tickets, and a central computer system that maintains a reference database for the tariff. Use cases should include BuyOneWayTicket, BuyWeeklyCard, BuyMonthlyCard, and UpdateTariff. Also include the following exceptional cases: TimeOut (i.e., traveler took too long to insert the right amount), TransactionAborted (i.e., traveler selected the cancel button without completing the transaction), DistributorOutOfChange, and DistributorOutOfPaper.

Exercise

- What is the difference between a scenario and a use case? When do you use each construct?
- Consider an ATM system. Identify at least three different actors that interact with this system.
- Draw a class diagram representing a book defined by the following statement: “A book is composed of a number of parts, which in turn are composed of a number of chapters. Chapters are composed of sections.” Focus only on classes and relationships.
- Consider the process of ordering a pizza over the phone. Draw an activity diagram representing each step of the process, from the moment you pick up the phone to the point where you start eating the pizza. Do not represent any exceptions. Include activities that others need to perform.